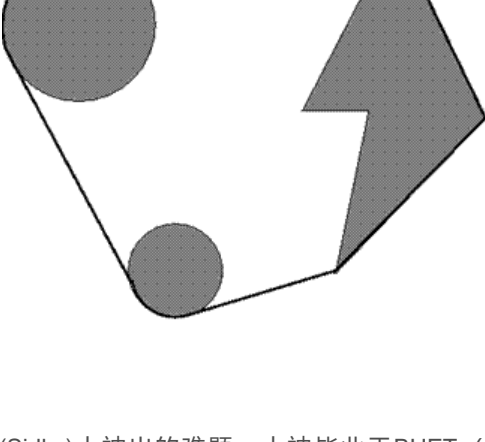


题目链接

UVa11726 - Crime Scene

题意

给定n (n≤100) 个物体，每个物体都是一个圆或者k (k≤10) 边形，用长度尽量小的绳子把它们包围起来。



分析

孟加拉国Manzurur Rahman Khan (Sidky)大神出的难题，大神毕业于BUET（Bangladesh University of Engineering and Technology）。

几何图形集合的凸包问题（凸包有两个最优性：长度最短；面积最小），本题的基本图形有圆，所以可以这么做：

求出所有点和圆的切点，然后所有的点和原有的点一起做一次凸包，求出凸包的边长，如果连续的两个点都是同一个圆上的切点，计算长度的方式就要变成求弧长。

这里不说取巧的方法，而是详细探讨几何图形集合的凸包问题的一般求法，姑且叫**滚边法**吧：在所有图形中找到最低点，多个图形包含最低点时任取一个作为起点即可，初始时假想从起点水平往右的向量作为当前向量，找出逆时针最小旋转量使其刚好贴到下一个图形某点上，下一个图形如果是圆，则可以绕着切点逆时针转一定角度（注意：可能会转超过180度）再贴到其他图形的某一点上，依此不停滚边操作，直到回贴到起点。

接下来讲一些细节，先看这一份数据：

```
10
2
c 0 0 2
p 4 -1 2 1 2 1 -2 -1 -2
1
c 0 0 3
2
c 0 0 2
c 10 10 1
5
c 0 0 1
c 10 0 1
c 0 10 1
c 10 10 1
p 4 4 4 5 5 6 4 5 8
3
c 0 0 2
c 2 2 2
p 3 -1 2 1 4 3 -1
3
p 3 -2 5 2 5 4 8
c 5 -8 7
c -10 -1 9
4
p 2 -1 -8 -9 10
p 1 -5 -7
c -1 2 9
c 7 -7 1
3
p 2 9 2 -1 -5
p 4 9 -4 8 -4 -4 7 -3 -7
c 2 3 7
5
p 2 5 2 -1 -9
c 9 -7 8
p 2 -9 9 -2 9
c -4 1 8
p 3 -10 -9 4 3 -7 -9
6
p 1 1 -10
p 1 2 5
p 1 -5 2
c 3 -1 1
p 7 8 -4 -8 9 -1 10 -4 -5 -6 9 -9 -6 -9 -1
c -3 -6 4
```

其正确输出应该是：

```
Case #1: 13.148009
Case #2: 18.849556
Case #3: 37.779789
Case #4: 46.283185
Case #5: 18.749469
Case #6: 87.092793
Case #7: 63.571176
Case #8: 51.636719
Case #9: 84.282670
Case #10: 59.374851
```

以上结果可以利用可视化脚本画图再手算检验。

后面几条数据容易出错，调试发现坑在double精度及反三角函数acos这里，acos函数传参必须严格在[-1,1]区间，否则会得到nan，因此可以这样处理：

```
double safe_acos(double v) {
    return acos(min(max(v, -1.), 1.));
}
```

但更推荐另外一种方式：计算余弦值的过程中用long double保精度因此能严格在[-1,1]区间，这样就可以直接调用acos了（可以将下面AC代码里面的long double全部换成double再提交验证，会WA）。

AC代码

```
#include <iostream>
#include <iomanip>
#include <cmath>
using namespace std;

#define eps 1e-10

struct Point {
    double x, y;
    Point(double x = 0., double y = 0.): x(x), y(y) {}
};
typedef Point Vector;

Vector operator- (const Vector& A, const Vector& B) {
    return Vector(A.x - B.x, A.y - B.y);
}

int dcmp(double x) {
    return abs(x) < eps ? 0 : (x < 0. ? -1 : 1);
}

bool operator== (const Point& a, const Point& b) {
    return dcmp(a.x - b.x) == 0 && dcmp(a.y - b.y) == 0;
}

double Cross(const Vector& A, const Vector& B) {
    return A.x * B.y - A.y * B.x;
}

double Dot(const Vector& A, const Vector& B) {
    return A.x * B.x + A.y * B.y;
}

double Lsq(const Vector& A) {
    return A.x*A.x + A.y*A.y;
}

#define N 102
Point c[N], p[11*N]; double r[N], pi2 = M_PI + M_PI; int n; bool esc0[N], esc[11*N];

double solve() {
    cin >> n;
    int a = 0, b = 0;
    while (n--) {
        char ch; cin >> ch;
        if (ch == 'p') {
            int k; cin >> k;
            while (k--) cin >> p[b].x >> p[b].y, esc[b++] = false;
        } else cin >> c[a].x >> c[a].y >> r[a], esc0[a++] = false;
    }
    for (int i=0; i<a; ++i) if (!esc0[i]) for (int j=i+1; j<a; ++j) {
        if (dcmp(r[i]-r[j]) <= 0 && dcmp(sqrt(Lsq(c[i]-c[j])) - r[j]+r[i]) <= 0) {
            esc0[i] = true; break;
        }
        if (dcmp(r[i]-r[j]) > 0 && dcmp(sqrt(Lsq(c[i]-c[j])) - r[i]+r[j]) <= 0) esc0[j] = true;
    }
    for (int i=0; i<b; ++i) if (!esc[i]) {
        for (int j=i+1; j<b; ++j) if (p[i] == p[j]) {
            esc[i] = true; break;
        }
        if (esc[i]) continue;
        for (int j=0; j<a; ++j) if (dcmp(sqrt(Lsq(p[i]-c[j])) - r[j]) <= 0) {
            esc[i] = true; break;
        }
    }
    for (int j=a, i=a=0; i<j; ++i) if (!esc0[i]) c[a] = c[i], r[a++] = r[i];
    for (int j=b, i=b=0; i<j; ++i) if (!esc[i]) p[b++] = p[i];
    if (a == 1 && b == 0) return pi2 * r[0];

    if (a == 0 && b < 2) return 0.;
    int s = 0; bool f = a>0;
    for (int i=0; i<a; ++i) if (c[i].y-r[i] < c[s].y-r[s]) s = i;
    for (int i=0; i<b; ++i) {
        if (f) {
            if (p[i].y < c[s].y-r[s]) s = i, f = false;
        } else if (p[i].y < p[s].y) s = i;
    }

    double t = 0., g = 0.; int x = s; bool u = f; Vector v(1.);
    while (true) {
        int y = -1; bool q; long double h;
        if (u) {
            for (int i=0; i<a; ++i) if (i != x) {
                Vector z = c[i] - c[x]; long double l = sqrt(Lsq(z)), sin = (r[i]-r[x])/l, cos = sqrt(1.-sin*sin);
                Vector w(z.x*cos + z.y*sin, z.y*cos - z.x*sin);
                cos = dcmp(Cross(v, w)) < 0 ? -2. - Dot(v, w) / l : Dot(v, w) / l;
                if (y < 0 || dcmp(cos-h) > 0) y = i, h = cos, q = true;
            }
            for (int i=0; i<b; ++i) {
                Vector z = p[i] - c[x]; long double l = sqrt(Lsq(z)), sin = r[i]/l, cos = sqrt(1.-sin*sin);
                Vector w(z.x*cos + z.y*sin, z.y*cos - z.x*sin);
                cos = dcmp(Cross(v, w)) < 0 ? -2. - Dot(v, w) / l : Dot(v, w) / l;
                if (y < 0 || dcmp(cos-h) > 0) y = i, h = cos, q = false;
            }
            h = h < -1. ? pi2-acos(-2.-h) : acos(h);
            if (f && x==s && dcmp(g + h - pi2) >= 0) return t + (pi2-g)*r[s];
            t += h*r[x] + (q ? sqrt(Lsq(c[y] - c[x]) - (r[y]-r[x])*(r[y]-r[x])) : sqrt(Lsq(p[y] - c[x]) - r[x]*r[x]));
            if (!f && !q && y==s) return t;
            g += h; x = y; u = q; v.x = cos(g); v.y = sin(g);
        } else {
            for (int i=0; i<a; ++i) {
                Vector z = c[i] - p[x]; long double l = sqrt(Lsq(z)), sin = r[i]/l, cos = sqrt(1.-sin*sin);
                Vector w(z.x*cos + z.y*sin, z.y*cos - z.x*sin);
                if (dcmp(Cross(v, w)) < 0) continue;
                cos = Dot(v, w) / l;
                if (y < 0 || dcmp(cos-h) > 0) y = i, h = cos, q = true;
            }
            for (int i=0; i<b; ++i) if (i != x) {
                Vector w = p[i] - p[x];
                if (dcmp(Cross(v, w)) < 0) continue;
                long double cos = Dot(v, w) / sqrt(Lsq(w));
                if (y < 0 || dcmp(cos-h) > 0) y = i, h = cos, q = false;
            }
            h = acos(h);
            t += q ? sqrt(Lsq(c[y] - p[x]) - r[y]*r[y]) : sqrt(Lsq(p[y] - p[x]));
            if (!f && !q && y==s) return t;
            g += h; x = y; u = q; v.x = cos(g); v.y = sin(g);
        }
    }
    return t;
}

int main() {
    ios::sync_with_stdio(false); cin.tie(0); cout.tie(0);
    cout << fixed << setprecision(6);
    int t; cin >> t;
    for (int kase=1; kase<=t; ++kase) cout << "Case #" << kase << ": " << solve() << endl;
    return 0;
}
```